ELSEVIER

PII: S0167-4048(96)00002-8

Computers & Security, Vol. 15, No. 4, pp. 327–338, 1996
Copyright © 1996 Elsevier Science Ltd
Printed in Great Britain. All rights reserved
0167-4048/96 $15.00

refereed paper

An implementable secure voting scheme

Joan Borrell and Josep Rifà

Departament d'Informàtica, Universitat Autònoma de Barcelona, Edifici C-08193 Bellaterra, Spain (joan@cd.uab.es)

In this paper we propose a secure voting scheme that reduces the cryptographic and communication requirements in comparison with other schemes which have been presented. No special communication channels are needed, and therefore it can be easily implemented on any existing computer network. Secret and authentic communications are established between voters and the algorithms participating in the scheme. The ballot is administered by an election board that uses a secret sharing scheme among its members. Privacy of ballots is guaranteed by randomly spreading them into suitable sequences of values. Our scheme meets the commonly accepted security requirements and does not impose restrictions on voter behaviour. Copyright ©1996 Elsevier Science Ltd

Keywords: Electronic voting, Local area networks, Secret sharing schemes, Security requirements.

1. Introduction

Electronic voting schemes have been studied from many points of view, both theoretical and practical. A good review of the various articles published on this topic can be found in [1].

Among the recent schemes presented, [1–4] attempt to limit the volume of communications required so that a practical implementation is possible. Nevertheless, there continues to be a high degree of complexity involved.

In [2–4] the voters send their ballot through an anonymous channel (a channel which suppresses the origin of the messages). Among the anonymous channel schemes which have been proposed, the only one which could be implemented on current communication networks is the one proposed in [5], with the appropiate countermeasures mentioned in [6]. This scheme is based on K intermediate machines (*mixes*), in which each machine receives the messages (ballots) from the previous machine, suppresses their origin and sends them to the next machine in alphabetical order. Thus, $K+1$ communications are required to send a user's ballot to the ballot box. The integrity of these *mixes* must be guaranteed.

In the scheme outlined in [1], the anonymous channels are suppressed. Nevertheless, this scheme requires that a public bulletin board be used and that there be a committee of K members which performs basically the same function as the previous *mixes*. This scheme also requires $K+1$ communications in order to send the ballot from the user to the ballot box; in addition, the integrity of the public bulletin board must be maintained.

In this paper we present a secure electronic voting scheme which can be implemented over a computer network. We have adopted simplicity as a crucial design requirement; this has led us to employ only a limited number of cryptographic tools and proto-

cols in the process of elaborating our secure voting scheme. We have also reduced the number of cryptographic operations which the voters must perform and the volume of communications needed. Thus, only two communications are required to send the user's ballot to the ballot box. Moreover, we have also eliminated the need for special communication channels. Our scheme can be implemented over any reliable protocol of transport layer, e.g. the *Transmission Control Protocol* (TCP) of the Internet.

With regard to the computer network in which our scheme is to be implemented, we would like to mention a statement made by Neumann [7], in which the author analyses the basic criteria of confidentiality, integrity, availability, reliability and assurance for the information systems utilised in electronic voting:

"An assessment of the realizability of those security criteria leads to the conclusion that, operationally, many of them are inherently unsatisfiable with any meaningful assurance."

The control of all aspects external to the voting scheme itself is much easier in a local network which belongs to one organization that has a single security policy. In reality, this local network does not have to be confined to a limited geographical area [8]. In the case of a wide area network, the possibilities that the messages and equipment could be altered or manipulated are multiplied; moreover, it complicates the administration of the users' cryptographic keys.

The computer network where the voting takes place must contain an asymmetric cryptosystem infrastructure with certificates. In other words, it must be a network where there is a process which acts as a certification authority, by which the network users (voters) can obtain certificates regarding their public keys (see e.g. [9]). By using these certificates, all the elements participating in the ballot can establish secret and authentic communications between themselves.

The ballot is administered by an election board made up of n members, k of which form a qualified majority to initiate the processes of collecting and counting the ballots. The parameters n and k correspond to those of a secret sharing scheme which allows the presence of cheaters among the members of the board to be detected [10]. Only a dishonest coalition of k election board members could violate the privacy of the ballots. This problem, which is also found in [1], could be resolved by choosing the board in such a way that the probability of such a coalition is negligible.

The security of the voting scheme that we present is based mainly on the integrity of the algorithms used in the voting (shared key management, ballot collecting and counting). These algorithms do not contain any important information, which means that the voting or counting could be restarted if they were suddenly interrupted for some outside reason.

To guarantee the privacy of the ballots, we use polynomials over the field $\mathcal{Z}_p$ of integers modulus p, with p being a prime number. Each ballot is represented as an error randomly introduced into a sequence of values obtained from these polynomials. The counting of the ballots is equivalent to detecting these errors, interpreting them and making them public. Our scheme allows any voter behaviour (valid ballot, null ballot or abstention), as opposed to other schemes using anonymous channels which do not normally permit voters to abstain.

The rest of our paper is structured as follows: Section 2 presents the well-known cryptographic tools and protocols used in our scheme. Section 3 describes the different voting phases, along with the algorithms which control them. Finally, we analyse the security offered by the scheme in Section 4, using the definition of a secure scheme given in [2], which we feel is quite complete. Our scheme meets all the requirements of this definition.

2. Cryptographic tools and protocols

2.1. Certificates and certification authorities

When using asymmetric cryptography, a fundamental problem in open environments with partners who do not know each other is the question of the authenticity of public keys. At the time of verifying a digital signature, the verifier must be sure that the public key which he uses for the verification of the signature is the public key of the supposed signer, i.e. he needs a key-to-name binding which he can trust. Without additional measures each user would have to perform an out-of-band verification of the authenticity of each partner's public key before trusting it.

The complexity of this problem can be reduced by certifying public keys through a third party whom both signer and verifier trust. This third party, the so-called *certification authority* (CA), signs the user's public key and his name (plus some additional data like a period of validity) with its own secret key. This piece of data is called a *certificate*. The certificate can be verified with the public key of the certification authority. Only the public key of the certification authority must be trusted, thus reducing the number of public keys which the individual user has to trust to one. In large user populations a single certification authority is not sufficient. The public keys of CAs may be certified again by other CAs, creating tree or net structures of certification authorities.

A complete description of certificate's management (creation, revocation, ...), together with a set of tools to implement CAs, can be found in [9].

2.2. Secret and authentic communications

All the communications in our voting scheme must be secret and authentic. To achieve this goal, we establish these communications by using the *station-to-station* (STS) protocol, introduced in [11]. The STS protocol is a simple, efficient authenticated key exchange that consists of Diffie–Hellman key establishment [12], followed by an exchange of authentication signatures. STS provides the communicating parties with the assurance that they know each other's true identities. Moreover, these parties end up sharing a common key known only to them, which is used thereafter to provide privacy and data integrity. STS authentication is public-key based, and therefore certificates are essential to it.

2.3. Secret sharing schemes

A (k, n) secret sharing scheme is a protocol to divide a secret D into n shares $D_1, \ldots, D_n$ in such a way that:

(1) Knowledge of any k shares D_i is sufficient to reconstruct D efficiently.
(2) Knowledge of any $k-1$ shares D_i provides no more information about the value of D than was known before.

The first secret sharing scheme was introduced by Shamir [13]. Shamir's scheme is not secure against cheating. A participant in his scheme can send a false share D_i' and force the reconstruction of an incorrect secret D', without being detected.

A cornerstone in the administration of our voting scheme is the management of the ballot box key (K_b). We distribute this key among the members of the electoral board, according to a secret sharing scheme presented by Borrell and Rifà [10].

This scheme improves Shamir's scheme by allowing the presence of coalitions of cheaters to be detected. Therefore, it guarantees the correctness of the ballot box key reconstruction and the correctness of the ballot counting process.

3. Voting scheme

The voting scheme which we propose involves nine elements: the network administrator, network users

(voters), election board, shared key generation algorithm (SKGA), ballot collecting algorithm, shared key retrieving algorithm (SKRA), counting algorithm, ballot box and electoral roll. These algorithms are described in Figs 2, 4, 5 and 6 respectively. Figure 1 contains a general outline of all these elements including how they interact. Labels associated to the arcs indicate the order of communication.

The voting scheme has three phases: *voting preparation, ballot collecting* and *ballot counting.*

All the communications which take place between the algorithms, or between a given algorithm and the voters or members of the election board, are secret and authentic, following the STS protocol of [11]. This implies that the information exchanged between the communicating parties travels in encrypted form using a symmetric cryptosystem, with a session key known only to the parties involved. This encryption is not shown in the algorithms in order to make them easier to read.

A basic assumption which we have made in our voting scheme is that the four algorithms used are implemented on a secure device such as a "Smart Card". This does not allow them to be modified nor does it allow intermediate results to be accessed from their computations. In addition, these algorithms must allow verification by any voter, e.g. by distributing their source code and permitting them to be compiled freely. Furthermore, we assume that the electoral roll and ballot box are implemented as external files available to the public, whereas writing is restricted to the algorithms which control the voting.

None of the four algorithms contains any important information. The ballots and control information needed to retrieve the ballots are stored directly in the ballot box. The information about who has voted is stored in the electoral roll whereas information about the ballot box's key is spread among the members of election board in accordance with the shared key scheme outlined in Section 2.3.

3.1. Voting preparation phase

Initially, we assume that all the elements involved in the voting (voters, shared key algorithms, collecting algorithm and counting algorithm) have generated their own public and secret key pair and have obtained the appropriate certificate for their public key.

In this phase the network administrator prepares the electoral roll with all possible voters. Afterwards, the n members of the election board are selected from the roll and the composition of this board is made public. In this phase the administrator also receives the different voting options, assigns them a codification and makes them public.

The voting options are encoded in such a way that for option i there is a corresponding set $C_i \subset \mathcal{Z}_p$, $C_i \cap C_j = \emptyset, \forall i \neq j$.

The administrator then executes the four algorithms which control the voting. These algorithms contain the built-in synchronization mechanisms needed to execute freely, which means that the administrator is no longer needed. His intervention will only be required in the case that it becomes necessary to restart the voting because one of the algorithms has stopped suddenly.

All the algorithms are written assuming that there are j voters.

The SKGA (see Fig. 2) computes the ballot box key K_b, of a symmetric cryptosystem. [The following notation is used: $E_{K_b}(x)$ and $D_{K_b}(x)$ respectively indicate the encrypted and the decrypted form of x using the ballot box key.] It then computes the shares that must allow a subsequent recomputation of this key and distributes them among the members of the board. Immediately, it initializes the first two rows of the ballot box. The first row contains a series of values needed to compute the ballots, which are common for all voters. The second contains the control information needed to count the ballots. Finally, it communicates K_b to the ballot collecting

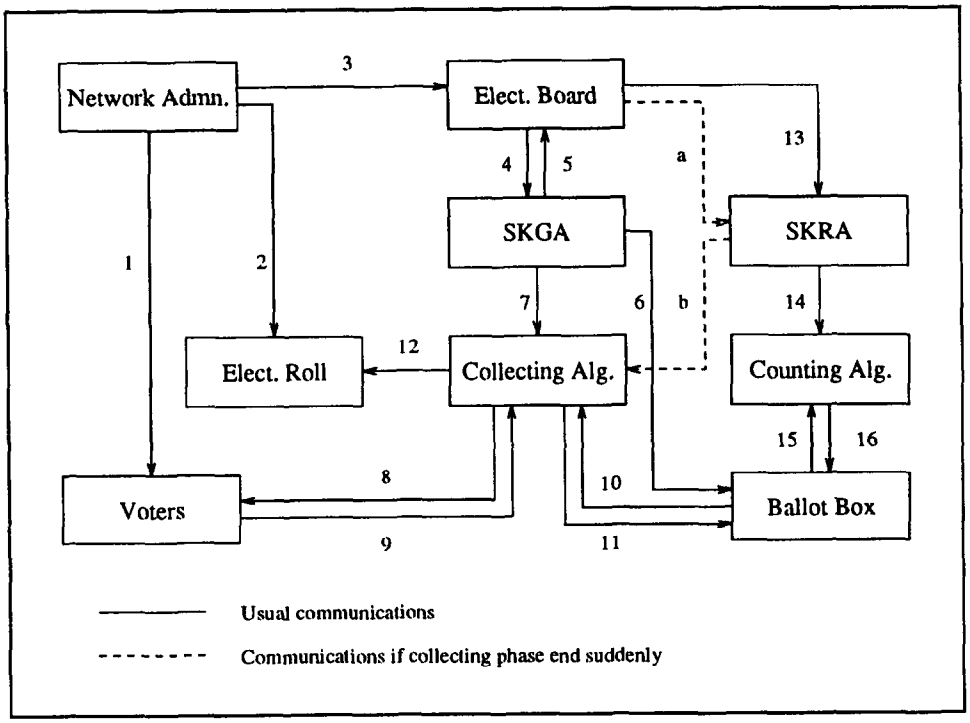


Fig. 1. Elements involved in the voting scheme.

algorithm, which allows the second phase of the voting to begin.

The contents of the ballot box at the end of the preparation phase is:

α_1	α_m	α_j
$E_{K_b}(0)$	$E_{K_b}(0)$	$E_{K_b}(0)$

3.2. Ballot collecting phase

The ballots are collected once the members of the election board have started the computation of key K_b of the ballot box and this key has been passed to the ballot collecting algorithm (see Fig. 4).

From Fig. 4, we can see that the method by which the voters cast their ballot is the following:

(1) The collecting algorithm reads the control information contained in the second row of the ballot box and decrypts it.
(2) Each voter V_t, after receiving $(p_{V_t}, \alpha_1, \ldots, \alpha_j, h_t)$ from the collecting algorithm, must compute $p_{V_t}(\alpha_{h_t})$ and add to it a value c_{ih_t} chosen at random from C_i, in accordance with the voting option desired.
(3) The ballot collecting algorithm receives $p_{V_t}(\alpha_{h_t})+c_{ih_t}$ and writes $p_{V_t}(\alpha_1), \ldots, p_{V_t}(\alpha_{h_t}) + c_{ih_t}, \ldots, p_{V_t}(\alpha_j)$ at the end of the ballot box.
(4) Afterwards, the collecting algorithm updates the value of the ballot box's second row.
(5) The contents of the ballot box with the first

```
begin
  repeat for all members of the election board M_i
  begin
    establish a secret and authentic communication with M_i (see [11])
    receive an input value m_i from M_i
    close the secret and authentic communication with M_i
  end
  generate key K_b of the ballot box by using m_i
  compute the shares D_i for K_b (see [10])
  repeat for all members of the election board M_i
  begin
    establish a secret and authentic communication with M_i
    send share D_i to M_i
    close the secret and authentic communication with M_i
  end
  for m = 1 ··· j do
  begin
    choose α_m ∈ Z_p at random
    write α_m to column m of the first arrow of the ballot box
  end
  initialize each column of the second row of the ballot box to E_{K_b}(0)
  establish a secret and authentic communication with the collecting algorithm
  send K_b to the collecting algorithm
  close the secret and authentic communication with the collecting algorithm
end (SKGA)
```

Fig. 2. Shared key generation algorithm (SKGA).

cast ballot is:

α_1	α_{h_1}	α_j
$E_{K_b}(p_{V_1}(\alpha_1))$	$E_{K_b}(p_{V_1}(\alpha_{h_1}))$	$E_{K_b}(p_{V_1}(\alpha_j))$
$p_{V_1}(\alpha_1)$	$p_{V_1}(\alpha_{h_1}) + c_{ih_1}$	$p_{V_1}(\alpha_j)$

(6) The collecting algorithm writes the value $E_{K_b}(h_t)$ to the entry corresponding to voter V_t in the electoral roll. This has two functions: to flag the voter as having voted and to permit the algorithm to distribute the h_t values without repeating among the different voters.

These last instructions must be executed in an *all or nothing* fashion in order to guarantee the integrity of data contained by the ballot box and the electoral roll. The *commitment, concurrency and recovery* (CCR) service element from the OSI application layer can be used to construct such an atomic action [14].

Each ballot has the following structure in the ballot box:

$$p_{V_t}(\alpha_1), \ldots, p_{V_t}(\alpha_{h_t}) + c_{ih_t}, \ldots, p_{V_t}(\alpha_j).$$

Voter V_t, who knows p_{V_t}, h_t and c_{ih_t}, can verify that his ballot has been correctly entered into the ballot box.

Assuming that there are g voters out of the j possible, the contents of the ballot box at the end of the collecting phase is shown in Fig. 3.

The second row of the ballot box ends up containing the sum of all the polynomials p_{V_t}, computed for each value α_m and encrypted in accordance with K_b, without the values c_{ih_t} added by the voters. Considering the c_{ih_t} as errors introduced by the voters, the second row functions as control information so that these errors can be detected and the ballots can be subsequently counted.

If the ballot collecting phase ends properly, at least k members of the board will communicate with the SKRA (see Fig. 5) and will transmit their shares to it. This algorithm, after checking that there are no cheaters among these members, will compute key K_b and will transfer it to the counting algorithm.

If the ballot collecting phase ends suddenly for any outside reason (maliciously or not), the network administrator will re-execute the collecting and counting algorithms and the SKRA. The SKRA will rebuild key K_b in the same way as is described in the previous point and will send it to the ballot collecting algorithm. Given that the collecting algorithm only needs key K_b and the information contained in the ballot box and the electoral roll, the voting can continue without any problem.

3.3. Ballot counting phase

The ballots are counted once the voting has finished (i.e. when everyone has voted or the voting deadline has been reached) and once the election board members have started the computation of the ballot box key K_b and this key has been passed to the counting algorithm (see Fig. 6).

Assuming that there are g voters out of the j possible, the counting algorithm begins to retrieve the control information contained in the second row of the ballot box. Afterwards, it continues to add up each column of the next g rows in the ballot box. The obtaining of the ballots goes on to compute the difference between these sums and the results of decrypting the control information for each column. Finally, it decodes this difference and writes the result at the end of the ballot box itself.

The counting algorithm only needs key K_b and the ballot box information. This means that the administrator can re-execute the counting and shared key retrieving algorithms if the counting phase stops suddenly and the process can begin again without any problem.

4. Security of our scheme

Reference [2] contains a definition of the security of a voting scheme which we consider complete. This definition is based on the completion of a series of security requirements. Below we review these requirements along with an indication of how our scheme fulfils each one.

Reference [2] also contains some other implicit security requirements which no voting scheme can guarantee of itself (ours included), e.g. the integrity of the electoral roll and ballot box. In reference to all these issues we again mention [7]. We also assume that an acceptable degree of security is possible in a controlled environment such a local area network.

- *Completeness:* all valid votes are counted correctly. If we assume the integrity of the collecting algorithm, then all the ballots collected are written to the public file that serves as a ballot box. In addition, each voter can check that his ballot has actually been entered into the ballot box. Given the assumption of the integrity of the counting algorithm, all the ballots in the

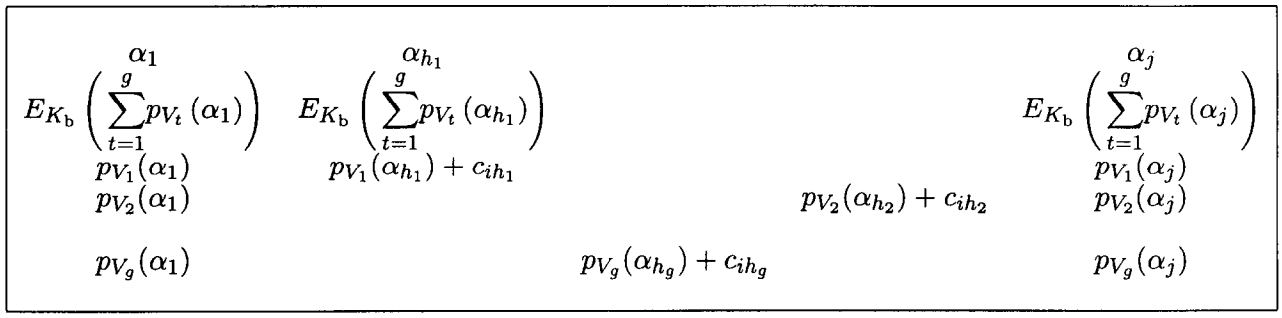


Fig. 3. Contents of the ballot box at the end of the collecting phase.

ballot box are counted correctly and the result is also written to the ballot box.

- *Soundness:* the dishonest voter cannot disrupt the voting. Voters acts only on the value $p_{V_t}(\alpha_{h_t})$ that is sent to them by the collecting algorithm, by adding a given value c_{ih_t} to it. They do not act on any other $p_{V_t}(\alpha_m)$ values. Therefore, a dishonest voter could only act fraudulently with respect to his own ballot, by choosing an invalid c_{ih_t}. This action is reflected as a null ballot and it will not affect the voting. Furthermore, the collecting algorithm has sufficient time controls to prevent it from waiting indefinitely while a voter maliciously tries to delay his ballot.

- *Privacy:* all votes must be secret. The secrecy of the ballots is protected by the fact that any possible spy does not know the values of p_{V_t} and h_t. Polynomial p_{V_t} is of degree $j+1$ and therefore it cannot be computed from the values $p_{V_t}(\alpha_m)$, $m = 1 \ldots j$ written to the ballot box. Nor is the column h_t which is assigned to each voter known by anyone except the voter himself, and thus it is not possible to relate the ballots to the voters even after the counting has taken place. In a normal situation the decrypted value of the second row of the ballot box, i.e. the one used as a control when retrieving the ballots, is not made public. Nor is the value K_b known and thus privacy is guaranteed. The only serious threat to the voters' privacy stems from a dishonest coalition of k members of the election board, who might obtain the key K_b on their own. It is necessary to choose the election board members in such a way that the probability of such a coalition is negligible. This can be done, for example, by choosing the board members from groups with opposing interests.

- *Unreusability:* no voter can vote twice. Assuming the integrity of the collecting algorithm and of the electoral roll, no-one can vote twice since the algorithm checks the roll before allowing any ballot to be cast.

- *Eligibility:* no-one who isn't allowed to vote can vote. For the same reasons as in the previous case.

- *Fairness:* the counting of ballots does not affect the voting. In a normal situation counting does not begin until the ballot collecting phase has been completed and the shared key retrieving algorithm has recomputed the key K_b and sent it to the counting algorithm. This means that the counting takes place after the ballot collection and cannot influence the voting. The only way to perform a partial counting before completing the ballot collecting process consists of fraudulently obtaining key K_b. This can only

```
A, B : array[1 ··· j] of elements of Z_p
begin
  repeat
    try to establish a secret and authentic communication
                with the SKGA or the SKRA
  until this communication is possible
  receive key K_b from the SKGA or the SKRA
  close the secret and authentic communication with the SKGA or the SKRA
  for m = 1 ··· j do A[m] ← α_m from the first row of the ballot box
  repeat for all voters V_t, or until the end of voting time
  begin
    establish a secret and authentic communication with V_t
    receive the petition to vote of V_t
    check that V_t is on the electoral roll and that he has not already voted
    if V_t can vote then
    begin
      for m = 1 ··· j do B[m] ← D_{K_b}(E_{K_b}(∑ p_{V_t}(α_m)))
                  (from the column m of the second row of the ballot box)
      choose h_t ∈ {1 ··· j} at random, without repetition
      generate a polynomial p_{V_t} at random, with degree of p_{V_t} = j + 1
      send (p_{V_t}, α_1, ..., α_j, h_t) to V_t
      wait for response p_{V_t}(α_{h_t}) + c_{ih_t} from V_t for a prefixed time
      if the response has arrived then
      perform the following instructions as a CCR atomic action [14]
        write p_{V_t}(α_1), .., p_{V_t}(α_{h_t}) + c_{ih_t}, .., p_{V_t}(α_j) at the end of the ballot box
        for m = 1 ··· j B[m] ← B[m] + p_{V_t}(α_m)
        for m = 1 ··· j write E_{K_b}(B[m]) in the second row of the ballot box
        write E_{K_b}(h_t) in the V_t entry in the electoral roll
                  to indicate that V_t has already voted
      end (CCR atomic action)
    end
    close the secret and authentic communication with V_t
  end
end (collecting)
```

Fig. 4. Ballot collecting algorithm.

```
begin
  repeat until there are no cheaters
  begin
    repeat for at least k members of the election board
    begin
      establish a secret and authentic communication with M_i
      receive the share D_i from M_i
      close the secret and authentic communication with M_i
    end
  verify the non-existence of cheaters among these members
              of the election board (see [10])
  end
  compute K_b from the shares received
  try to establish a secret and authentic communication with
              the collecting algorithm
  if this communication has been established then
  begin
    send K_b to the collecting algorithm
    close the secret and authentic communication with the collecting algorithm
  end
  else
  begin
    establish a secret and authentic communication with the counting algorithm
    send K_b to the counting algorithm
    close the secret and authentic communication with the counting algorithm
  end
end (SKRA)
```

Fig. 5. Shared key retrieving algorithm (SKRA).

be done by a dishonest coalition of k members of the election board. In this regard, the same measures should be taken as those mentioned when addressing the privacy issue.

Verifiability: no-one can falsify the result of the voting. The result of the voting is written by the counting algorithm to the ballot box and it can be accessed by anyone. Assuming the integrity of the counting algorithm and of the ballot box itself, no-one can falsify this result. We could also assume that a coalition of k members of the election board could obtain the key K_b on its own in order to perform an independent counting. But, even in this case, the results cannot be falsified since any independent counting could not be written to the ballot box. The counting could be stopped if the shared key retrieving algorithm were not able to communicate with any honest group of at least k election board members in order to retrieve K_b. But in such a serious situation of decline, perhaps it would be necessary to cancel the entire process...

A, B, C : array$[1 \cdots j]$ of elements of $\mathcal{Z}_p$
begin
  repeat
    try to establish a secret and authentic communication with the SKRA
  until this communication is possible
  receive key K_b from the SKRA
  close the secret and authentic communication with the SKRA
  from the second row of the ballot box, for $m = 1 \cdots j$ do
    $A[m] \leftarrow E_{K_b}(\sum_{t=1}^{g} p_{V_t}(\alpha_m))$
  from the rows $3 \cdots g + 2$ of the ballot box, for $m = 1 \cdots j$ compute
    $B[m] \leftarrow \sum_{t=1}^{g} p_{V_t}(\alpha_m) + c_{im}$
  for $m = 1 \cdots j$ compute
    $C[m] \leftarrow B[m] - D_{K_b}(A[m]) = c_{im}$
  for $m = 1 \cdots j$ do
    if $C[m] = 0$ write "*Abstention*" at the end of the ballot box
    else
      if $C[m]$ is a valid codification write the vote
                associated to $C[m]$ at the end of the ballot box
      else write "*Null ballot*" at the end of the ballot box
end (counting)

Fig. 6. Counting algorithm.

5. Conclusion

This paper has proposed a secure voting scheme that reduces the cryptographic and communication requirements in comparison with other schemes which have been presented. No special device or communications channels are needed, meaning that it can be easily implemented on any existing computer network. Moreover, the voting process can be interrupted and restarted afterwards without affecting the voting. Our scheme meets the commonly accepted security requirements and does not impose restrictions on voter behaviour.

The voting scheme described in this paper is currently under implementation over the local area network of our department. We expect to have a working prototype in late spring 1997.

Acknowledgements

This work has been partially funded by the Spanish Government Commission CICYT through its project No. TIC94-0331.

References

[1] T. Okamoto, A. Fujioka and K. Ohta, A practical large scale secret voting scheme based on non-anonymous channels, *Proceedings of SCIS93*, Paper 1C, Shuzenji, Japan (Jan. 1993).

[2] A. Fujioka, T. Okamoto and K. Ohta, A practical secret voting scheme for large scale elections, *LNCS 718, Auscrypt'92*, pp. 244–251, Springer, Berlin (1992).

[3] K. Sako, Electronic voting scheme allowing open objection to the tally, *IEICE Trans. Fund. ECCS*, E77-A (1) (1994) 24–30.

[4] K. Sako and J. Kilian, Receipt-free mix-type voting scheme—A practical solution to the implementation of a voting booth,

LNCS 921, Eurocrypt'95, pp. 393–403, Springer, Berlin (1995).

[5] D. Chaum, Untraceable electronic mail, return addresses, and digital pseudonyms, *Communications of the ACM*, 24 (1981) 84–88.

[6] B. Pfitzmann, A. Pfitzmann, How to break the direct RSA-implementation of mixes, *LNCS 434, Eurocrypt'89*, pp. 373–381, Springer, Berlin (1989).

[7] P.G. Neumann, Security criteria for electronic voting, *Proceedings of the 16th National Computer Security Conference*, pp. 478–482, Baltimore MD (Sept. 1993).

[8] K.E. Kirkpatrick, Why is a LAN a LAN?, *LNCS 396, Local Area Network Security*, pp. 3–4, Springer, Berlin (1989).

[9] W. Schneider (Ed.), *SecuDE Version 4.4 Documentation*, GMD, Darmstadt (Mar. 1995).

[10] J. Borrell and J. Rifà, Two robust secret sharing schemes, *Proceedings of the II Mediterranean Workshop on Coding and Information Integrity*, pp. 17–24, Mallorca (Feb. 1996).

[11] W. Diffie, P.C. Van Oorschot and M.J. Wiener, Authentication and authenticated key exchanges, *Designs, Codes and Cryptography*, 2 (1992) 107–125.

[12] W. Diffie and M.E. Hellman, New directions in cryptography, *IEEE Transactions on Information Theory*, 22 (1976) 644–654.

[13] A. Shamir, How to share a secret, *Communications of the ACM*, 22 (1979) 612–613.

[14] F. Halsall, *Data Communications, Computer Networks and Open Systems*, 4th edn, Addison-Wesley, Reading, MA (1996).